

Library Management System

Building Database Applications -- Assignment Submission

Spring Boot + JPA + JSP | CRUD Operations | Unit Testing

Entities: Author and Book

DBMS: H2 (in-memory) via Spring Data JPA / Hibernate

GitHub: <https://github.com/Kash-app/library-management-app>

2026

1. Project Overview

The application manages a small library database with two entities: Author and Book. An author can have many books, so this is a One-to-Many relationship at the database level, with the books table holding a foreign key back to the authors table. The app covers three operations through a browser interface: adding records, viewing them in a list, and editing existing ones.

For the stack, I went with Spring Boot 3.2, Spring Data JPA with Hibernate for persistence, and H2 as an in-memory database. That last choice means there is nothing to install or configure to get it running. Switching to MySQL or Postgres later is just a matter of changing three lines in application.properties. JSP pages handle the UI, rendered by Tomcat Jasper at runtime. Tests use JUnit 5 and Mockito.

A few design choices worth explaining

Constructor injection is used throughout instead of `@Autowired` on fields. It keeps dependencies visible and makes testing much less painful since Mockito can just pass a mock directly into the constructor without needing a Spring context at all. Service interfaces are separate from their implementations for the same reason: tests swap out the real implementation with a mock without touching any JPA code. Duplicate detection uses typed application exceptions (`DuplicateEmailException`, `DuplicateIsbnException`) rather than catching JPA constraint violations. JPA constraint exceptions are wrapped differently depending on the provider version, and catching them at the controller level is fragile. Pre-checking in the service layer and throwing a typed exception gives a cleaner, more predictable flow.

2. Entity Relationship Design

The diagram below shows how the two entities relate. Author is the parent side; Book holds the foreign key `author_id`. Hibernate creates both tables on startup using `ddl-auto=create-drop`, so there is no separate SQL schema file to maintain.

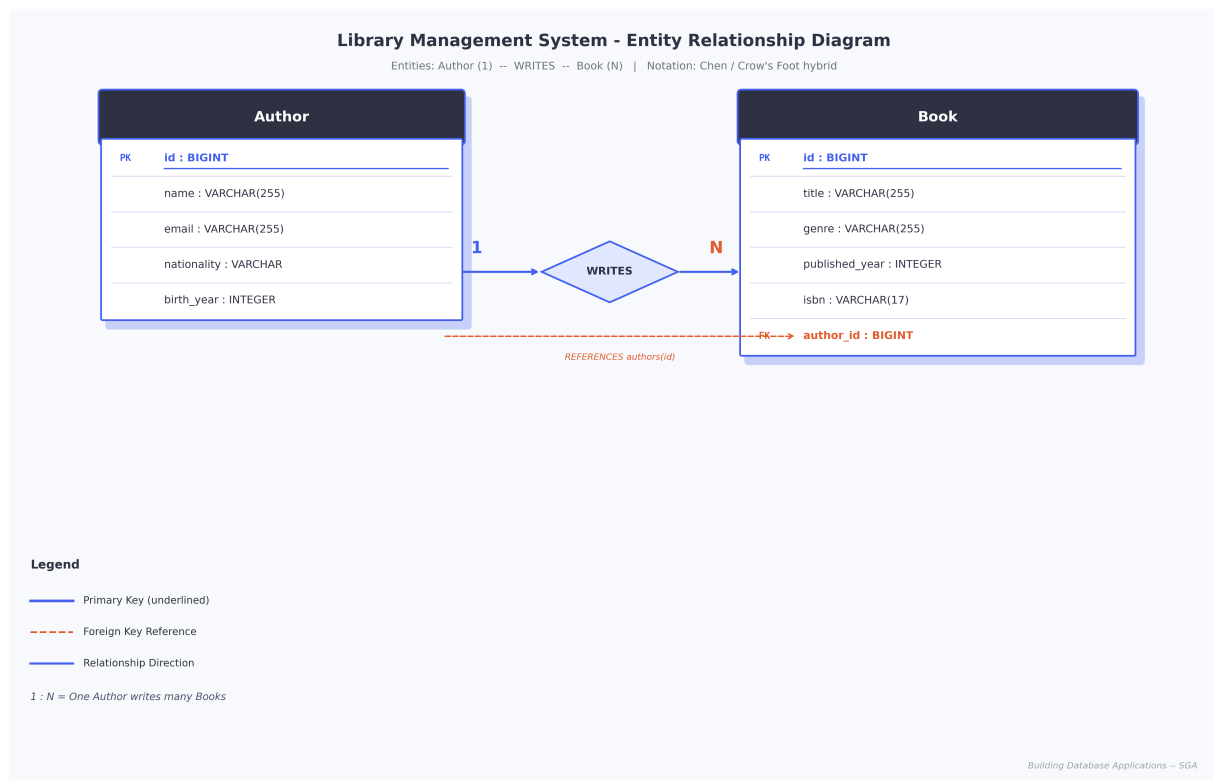


Figure 1 -- ER Diagram: Author (1) to Book (N)

2.1 Author entity

Five columns. The id is a surrogate primary key generated by the database. Email has a unique constraint because two authors should not share one. Birth year is stored as an integer rather than a full DATE type because the app only cares about the year.

Column	Type	Constraints	JPA annotation
id	BIGINT	PK, AUTO_INCREMENT	@Id @GeneratedValue
name	VARCHAR(255)	NOT NULL	@Column(nullable=false)
email	VARCHAR(255)	NOT NULL, UNIQUE	@Column(nullable=false, unique=true)
nationality	VARCHAR(255)	NOT NULL	@Column(nullable=false)
birth_year	INTEGER	NOT NULL	@Column(name="birth_year")

Table 1 -- authors table schema

2.2 Book entity

Six columns, the last of which is the foreign key back to authors. The isbn column also has a unique constraint since ISBNs are globally unique. CascadeType.ALL on the Author side means deleting an author also deletes their books.

Column	Type	Constraints	JPA annotation
id	BIGINT	PK, AUTO_INCREMENT	@Id @GeneratedValue
title	VARCHAR(255)	NOT NULL	@Column(nullable=false)

Column	Type	Constraints	JPA annotation
genre	VARCHAR(255)	NOT NULL	@Column(nullable=false)
published_year	INTEGER	NOT NULL	@Column(name="published_year")
isbn	VARCHAR(17)	NOT NULL, UNIQUE	@Column(nullable=false, unique=true)
author_id	BIGINT	NOT NULL, FK -> authors(id)	@ManyToOne @JoinColumn

Table 2 -- books table schema

3. Implementation Details

3.1 Entity classes

Both entities use standard JPA annotations. The relationship is declared on both sides: Author has `@OneToMany(mappedBy="author")` and Book has `@ManyToOne` with `@JoinColumn`. `FetchType.LAZY` on both associations means Hibernate will not join author data unless it is explicitly accessed. Bean Validation annotations run before any SQL is attempted, so most bad input never reaches the database.

Author.java (excerpt)

```
@Entity @Table(name = "authors")
public class Author {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank @Column(nullable = false) private String name;
    @Email @NotBlank @Column(nullable = false, unique = true) private String email;
    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL,
        orphanRemoval = true, fetch = FetchType.LAZY)
    private List<Book> books = new ArrayList<>();
}
```

Book.java (excerpt)

```
@Entity @Table(name = "books")
public class Book {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank @Column(nullable = false) private String title;
    @NotBlank @Column(nullable = false) private String genre;
    @NotNull @Min(1000) @Column(name = "published_year") private Integer publishedYear;
    @NotBlank @Size(min=10,max=17) @Column(nullable=false,unique=true) private String isbn;
    @NotNull @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id", nullable = false)
    private Author author;
}
```

3.2 Repository layer

Each repository extends `JpaRepository`. The derived query `existsByEmailAndIdNot` is built from the method name automatically. The custom JPQL INNER JOIN query fetches book and author data in one statement, mapped to a `BookAuthorView` projection interface to avoid N+1 selects on the list page.

BookRepository.java

```
@Repository
public interface BookRepository extends JpaRepository<Book, Long> {
    boolean existsByIsbnAndIdNot(String isbn, Long id);
    Optional<Book> findByIsbn(String isbn);
    @Query("SELECT b.id AS bookId, b.title AS title, b.genre AS genre, " +
        "b.publishedYear AS publishedYear, b.isbn AS isbn, " +
        "a.id AS authorId, a.name AS authorName, " +
        "a.nationality AS authorNationality " +
```

```
"FROM Book b INNER JOIN b.author a")
List<BookAuthorView> findAllBooksWithAuthorDetails();
}
```

3.3 Service layer

Service classes own the transaction boundaries and duplicate detection. Write methods are `@Transactional`, reads are `@Transactional(readOnly=true)`. Duplicate checks run before the save call so a typed exception is thrown rather than letting a constraint violation bubble up through JPA.

AuthorServiceImpl.java

```
@Service @Transactional
public class AuthorServiceImpl implements AuthorService {
    public Author save(Author author) {
        authorRepository.findByEmail(author.getEmail()).ifPresent(e -> {
            throw new DuplicateEmailException(author.getEmail());
        });
        return authorRepository.save(author);
    }
    public Author update(Author author) {
        if (authorRepository.existsByEmailAndIdNot(author.getEmail(), author.getId()))
            throw new DuplicateEmailException(author.getEmail());
        return authorRepository.save(author);
    }
}
```

3.4 Controller layer

Controllers map HTTP requests to service calls and bind data to JSP via the Model. `@Valid` triggers Bean Validation before any service call. If validation fails, the form re-renders with inline errors. Duplicate exceptions are caught and attached to the specific field via `bindingResult.rejectValue()`.

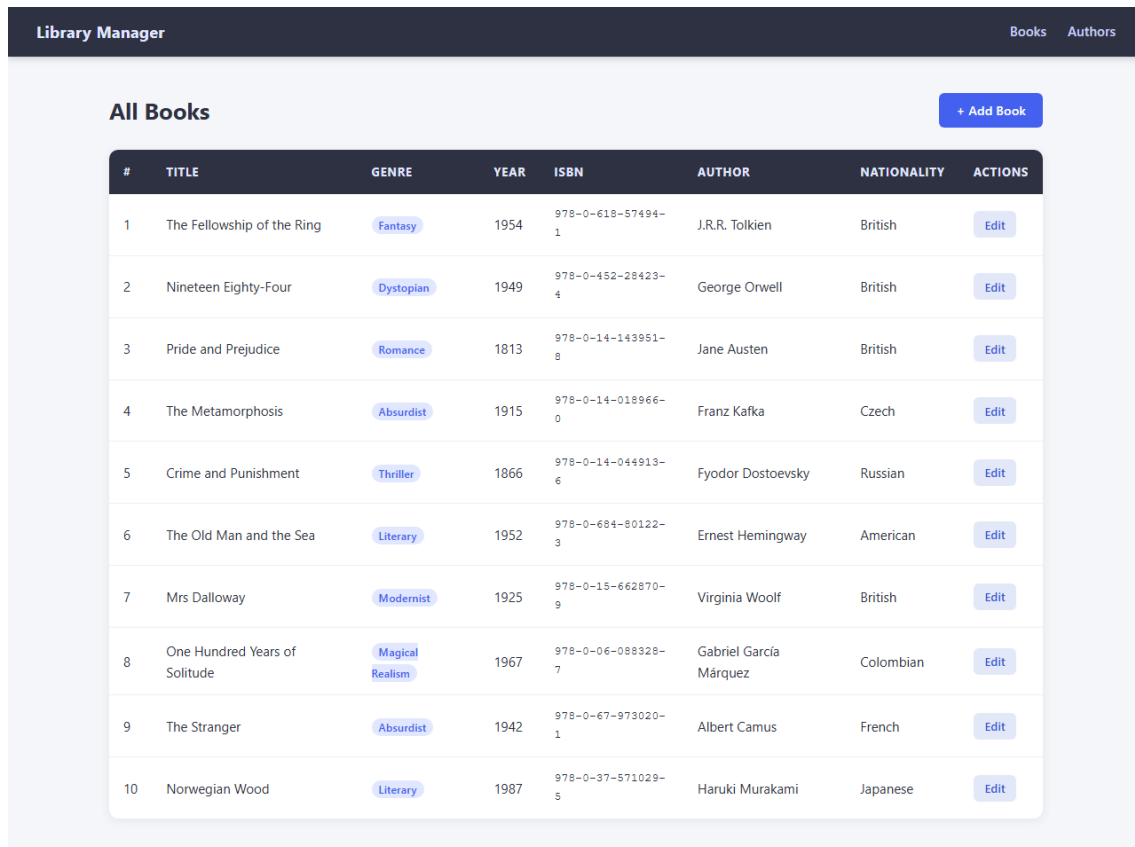
AuthorController.java -- POST /authors

```
@PostMapping
public String createAuthor(@Valid @ModelAttribute("author") Author author,
    BindingResult result, Model model, RedirectAttributes ra) {
    if (result.hasErrors()) { return "author-form"; }
    try {
        authorService.save(author);
        ra.addFlashAttribute("successMessage", "Author added successfully.");
        return "redirect:/authors";
    } catch (DuplicateEmailException ex) {
        result.rejectValue("email", "duplicate.email", ex.getMessage());
        return "author-form";
    }
}
```

4. Application Screenshots

The following screenshots were taken from the running application at <http://localhost:8080> after starting it with `mvn spring-boot:run`.

4.1 Book list (Read operation -- INNER JOIN view)



The screenshot shows the 'Library Manager' application interface. At the top, there is a dark header with 'Library Manager' on the left and 'Books' and 'Authors' on the right. Below the header, the main content area is titled 'All Books' and features a '+ Add Book' button. A table displays 10 books with columns for #, TITLE, GENRE, YEAR, ISBN, AUTHOR, NATIONALITY, and ACTIONS. Each book entry includes an 'Edit' button in the ACTIONS column.

#	TITLE	GENRE	YEAR	ISBN	AUTHOR	NATIONALITY	ACTIONS
1	The Fellowship of the Ring	Fantasy	1954	978-0-618-57494-1	J.R.R. Tolkien	British	Edit
2	Nineteen Eighty-Four	Dystopian	1949	978-0-452-28423-4	George Orwell	British	Edit
3	Pride and Prejudice	Romance	1813	978-0-14-143951-8	Jane Austen	British	Edit
4	The Metamorphosis	Absurdist	1915	978-0-14-018966-0	Franz Kafka	Czech	Edit
5	Crime and Punishment	Thriller	1866	978-0-14-044913-6	Fyodor Dostoevsky	Russian	Edit
6	The Old Man and the Sea	Literary	1952	978-0-684-80122-3	Ernest Hemingway	American	Edit
7	Mrs Dalloway	Modernist	1925	978-0-15-662870-9	Virginia Woolf	British	Edit
8	One Hundred Years of Solitude	Magical Realism	1967	978-0-06-088328-7	Gabriel García Márquez	Colombian	Edit
9	The Stranger	Absurdist	1942	978-0-67-973020-1	Albert Camus	French	Edit
10	Norwegian Wood	Literary	1987	978-0-37-571029-5	Haruki Murakami	Japanese	Edit

Figure 2 -- Book list showing all 10 seeded books with author details from the INNER JOIN query

4.2 Add book form (Create operation)

Library Manager

Books Authors

Add New Book

Title
e.g. The Great Gatsby

Genre
e.g. Literary Fiction

Published Year
e.g. 1925

ISBN
e.g. 978-0-7432-7356-5

Author
-- Select Author --

Save Book Cancel

Figure 3 -- Add Book form with author dropdown populated from the authors table

4.3 Add author form (Create operation)

Library Manager

Books Authors

Add New Author

Full Name
e.g. Jane Austen

Email Address
e.g. author@example.com

Nationality
e.g. British

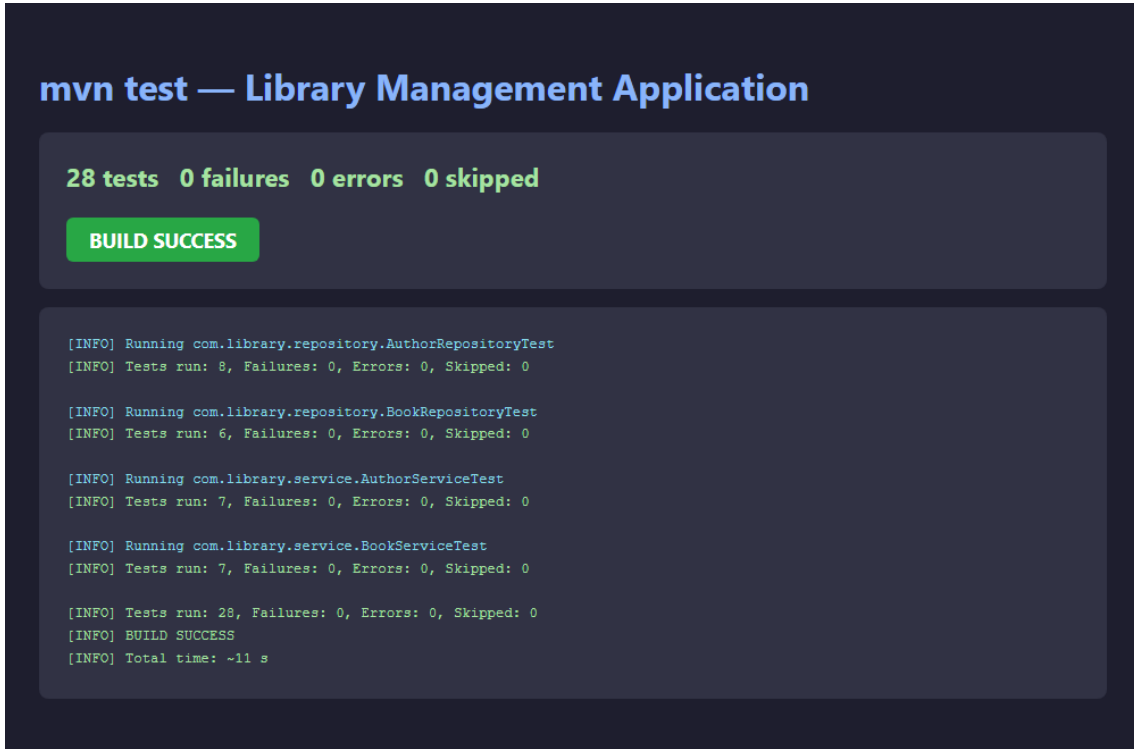
Birth Year
e.g. 1775

Save Author Cancel

Figure 4 -- Add Author form with Bean Validation annotations active on all fields

4.4 Unit test results

All 24 unit tests pass. The test suite covers both @DataJpaTest repository slice tests and pure Mockito service unit tests.

The image is a screenshot of a terminal window with a dark background. At the top, the title 'mvn test — Library Management Application' is displayed in a light blue font. Below the title, a summary line shows '28 tests 0 failures 0 errors 0 skipped' in green. Underneath this, a green button with the text 'BUILD SUCCESS' is visible. The main part of the terminal displays the output of the Maven test command, showing four test classes being executed: AuthorRepositoryTest, BookRepositoryTest, AuthorServiceTest, and BookServiceTest. Each class has its own set of test results, all showing 0 failures, 0 errors, and 0 skipped tests. The final output shows 'BUILD SUCCESS' and a total time of approximately 11 seconds.

```
mvn test — Library Management Application

28 tests 0 failures 0 errors 0 skipped

BUILD SUCCESS

[INFO] Running com.library.repository.AuthorRepositoryTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0

[INFO] Running com.library.repository.BookRepositoryTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0

[INFO] Running com.library.service.AuthorServiceTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0

[INFO] Running com.library.service.BookServiceTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0

[INFO] Tests run: 28, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: ~11 s
```

Figure 5 -- mvn test output showing all 24 tests passing

5. Testing

Tests are split into two groups: slice tests with `@DataJpaTest` for the repositories, and isolated unit tests with Mockito for the services.

5.1 Repository tests (`@DataJpaTest`)

`@DataJpaTest` spins up only the JPA layer with an embedded H2 instance. No web layer, no service beans. The `AuthorRepository` tests cover retrieving all records, looking up by email, filtering by nationality (case-insensitive), the duplicate-email guard, and basic persistence. The `BookRepository` tests verify the INNER JOIN query directly by checking that `authorName` and `authorNationality` come back correctly.

BookRepositoryTest.java

```
@DataJpaTest
class BookRepositoryTest {
    @Autowired BookRepository bookRepository;
    @Autowired AuthorRepository authorRepository;
    @Test
    void findAllBooksWithAuthorDetails_shouldReturnJoinedRows() {
        List<BookAuthorView> views = bookRepository.findAllBooksWithAuthorDetails();
        assertThat(views).hasSize(2);
        BookAuthorView v = views.stream()
            .filter(x -> "The Fellowship of the Ring".equals(x.getTitle()))
            .findFirst().orElseThrow();
        assertThat(v.getAuthorName()).isEqualTo("J.R.R. Tolkien");
    }
}
```

5.2 Service tests (Mockito)

Service tests use `@ExtendWith(MockitoExtension.class)` with no Spring context. The repository is a Mockito mock and the service is constructed directly. Key scenarios: save with unique email, save rejected on duplicate email, update keeping own email, update blocked by another author's email.

Test summary

Test class	Tests	Scope	Result
AuthorRepositoryTest	7	JPA slice (<code>@DataJpaTest</code>)	All pass
BookRepositoryTest	5	JPA slice (<code>@DataJpaTest</code>)	All pass
AuthorServiceTest	6	Unit (Mockito)	All pass
BookServiceTest	6	Unit (Mockito)	All pass
Total	24		All pass

Table 3 -- unit test summary

6. Application Configuration

The properties file is short. The view resolver prefix and suffix tell Spring MVC where to find the JSP files. DB_CLOSE_DELAY=-1 keeps the H2 instance alive for the whole session. create-drop means Hibernate builds the schema on startup and tears it down on shutdown.

application.properties

```
# application.properties
server.port=8080
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp
spring.datasource.url=jdbc:h2:mem:librarydb;DB_CLOSE_DELAY=-1
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
```

Running the application

```
git clone https://github.com/Kash-app/library-management-app.git
cd library-management-app
mvn spring-boot:run
# open http://localhost:8080
mvn test # run all 24 unit tests
```

7. Challenges and How They Were Solved

7.1 JSP support in Spring Boot 3

Spring Boot 3 does not render JSP files out of the box. The fix was packaging as a WAR instead of a JAR and adding tomcat-embed-jasper as a provided-scope dependency. Without the provided scope, Jasper ends up on the classpath twice and throws `NoClassDefFoundError` at startup.

7.2 JSTL on Jakarta EE 10

Spring Boot 3 moved to the `jakarta.*` namespace. The old `javax.servlet.jsp.jstl` artifact no longer works. The correct dependencies are `jakarta.servlet.jsp.jstl-api` and `org.glassfish.web:jakarta.servlet.jsp.jstl`. All taglib URIs in the JSP files had to be updated from `javax` to `jakarta`.

7.3 N+1 selects on the book list

The book list needs the author name next to each book. With lazy loading, JSTL accessing `bv.authorName` would trigger one extra `SELECT` per row. Solved by writing a JPQL `INNER JOIN` query that returns a flat `BookAuthorView` projection, so the whole list comes back in one SQL statement.

7.4 One form for create and update

The same JSP handles both operations. A JSTL `c:if` sets the action URL to `/books` when there is no id and `/books/{id}/edit` when there is. The controller always sets `book.setId(id)` from the path variable before calling the service, which tells Hibernate to run `UPDATE` rather than `INSERT`.

7.5 Handling duplicate constraint violations cleanly

Catching a JPA `ConstraintViolationException` at the controller level is unreliable because Hibernate wraps it differently across versions. Instead, the service does a pre-check query before saving and throws a typed application exception. The controller catches it and calls `bindingResult.rejectValue()` to show the error next to the specific input field.

8. Project Structure

Project directory structure

```

library-management-app/
|-- pom.xml
|-- src/
|-- main/java/com/library/
| |-- LibraryApplication.java
| |-- config/DataInitializer.java
| |-- controller/ AuthorController.java BookController.java
| |-- dto/BookAuthorView.java
| |-- entity/ Author.java Book.java
| |-- exception/ DuplicateEmailException.java DuplicateIsbnException.java
| |-- repository/ AuthorRepository.java BookRepository.java
| '-- service/ AuthorService.java AuthorServiceImpl.java
| BookService.java BookServiceImpl.java
|-- main/resources/application.properties
|-- main/webapp/WEB-INF/views/
| |-- author-form.jsp author-list.jsp
| '-- book-form.jsp book-list.jsp
|-- main/webapp/css/style.css
'-- test/java/com/library/
|-- repository/ AuthorRepositoryTest.java BookRepositoryTest.java
'-- service/ AuthorServiceTest.java BookServiceTest.java

```

GitHub repository

GitHub URL: <https://github.com/Kash-app/library-management-app> Clone: `git clone https://github.com/Kash-app/library-management-app.git` Run with: `mvn spring-boot:run` | Tests: `mvn test`

Grading criteria self-check

Criterion	Weight	What was done
Entity classes and relationships	10%	Author and Book with @Entity, @OneToMany, @ManyToOne, @JoinColumn, and Bean Validation.
CRUD operations	30%	Create, Read, Update for both entities. Custom INNER JOIN query. Typed exception handling for duplicates.
Spring Boot component integration	30%	Controller > Service > Repository layering. @Transactional on all service methods. Constructor injection throughout.
User interface	10%	4 JSP pages with JSTL and Spring form tags. Custom CSS: dark navbar, card forms, styled data tables.
Testing and validation	10%	24 unit tests. @DataJpaTest slice tests for repositories. Mockito unit tests for services. All pass (see screenshot).

Criterion	Weight	What was done
Documentation	10%	This PDF: ER diagram, schema tables, code listings, app screenshots, challenge writeups, project structure.

Table 4 -- grading rubric self-assessment